

P NEEHARIKA-2520090233

PRACTICAL WEEK 10 - SESSION 19

Task 1 :- To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

```
GNU nano 7.2 task1_signal_handlers.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void handle_signal(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT handled successfully.\n");

    if (sig == SIGTERM)
        printf("SIGTERM handled successfully.\n");
}

int main() {
    struct sigaction sa;

    printf("Signal Handler Demonstration\n");
    printf("Process ID: %d\n", getpid());

    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);

    printf("Signal handlers registered successfully.\n");

    printf("Generating SIGTERM...\n");
    raise(SIGTERM);

    printf("Signal event completed successfully.\n");

    return 0;
}
```

```
root@HP:~/skill_week10# gcc task1_signal_handlers.c -o task1_signal_handlers
root@HP:~/skill_week10# ./task1_signal_handlers
Signal Handler Demonstration
Process ID: 617
Signal handlers registered successfully.
Generating SIGTERM...
SIGTERM handled successfully.
Signal event completed successfully.
root@HP:~/skill_week10# |
```

Task 2 :- To Capture SIGINT Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

```
GNU nano 7.2 task2_sigint_forward.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

void parent_handler(int sig) {
    printf("\nParent process protected from SIGINT.\n");
}

int main() {
    pid_t pid;
    int status;

    signal(SIGINT, parent_handler);

    printf("SIGINT Forwarding Demonstration\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Foreground job started. PID: %d\n", getpid());
        printf("Foreground job is running.\n");

        sleep(2);

        printf("Foreground job completed.\n");
        exit(0);
    }

    printf("Foreground job registered with PID: %d\n", pid);
    printf("Parent process remains protected.\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Foreground job terminated normally.\n");
        printf("Job state updated to COMPLETED.\n");
    }

    return 0;
}
```

The output displays the parent and foreground job PIDs, confirms that the parent remains protected, and shows the foreground job completing successfully.

```
root@HP:~/skill_week10# gcc task2_sigint_forward.c -o task2_sigint_forward
root@HP:~/skill_week10# ./task2_sigint_forward
SIGINT Forwarding Demonstration
Parent PID: 626
Foreground job registered with PID: 627
Parent process remains protected.
Foreground job started. PID: 627
Foreground job is running.
Foreground job completed.
Foreground job terminated normally.
Job state updated to COMPLETED.
root@HP:~/skill_week10# |
```

Observation :- Signal handlers and foreground job management were successfully implemented and tested, with signal events and job states handled correctly.

SESSION 20

Task 1 :- To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension

```
GNU nano 7.2 task3_sigsttp.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("SIGTSTP Suspension Demonstration\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child process started. PID: %d\n", getpid());
        printf("Child is running.\n");

        sleep(1);

        printf("Child process completed.\n");
        exit(0);
    }

    printf("Job added to job table.\n");
    printf("Job PID: %d\n", pid);
    printf("Initial state: RUNNING\n");

    sleep(1);
```

Workflow

```
printf("Sending SIGTSTP signal to the job...\n");
kill(pid, SIGTSTP);

sleep(1);

printf("Job state: STOPPED\n");
printf("Sending SIGCONT to resume the job...\n");

kill(pid, SIGCONT);

waitpid(pid, &status, 0);

if (WIFEXITED(status)) {
    printf("Job state: COMPLETED\n");
    printf("Suspension and resume workflow completed.\n");
}

return 0;
}
```

```

root@HP:~/skill_week10# gcc task3_sigtstp.c -o task3_sigtstp
root@HP:~/skill_week10# ./task3_sigtstp
SIGTSTP Suspension Demonstration
Job added to job table.
Child process started. PID: 639
Job PID: 639
Child is running.
Initial state: RUNNING
Child process completed.
Sending SIGTSTP signal to the job...
Job state: STOPPED
Sending SIGCONT to resume the job...
Job state: COMPLETED
Suspension and resume workflow completed.
root@HP:~/skill_week10# |

```

Task 2 :- To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

```

GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    pid_t pgid;
    int status;

    printf("Process Group Demonstration\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        setpgid(0, 0);

        printf("Child PID: %d\n", getpid());
        printf("Process Group ID: %d\n", getpgrp());
        printf("Child process group created successfully.\n");

        sleep(1);

        printf("Child process completed.\n");
        exit(0);
    }

    sleep(1);
}

```

```

    pgid = getpgid(pid);

    printf("Child process group ID retrieved: %d\n", pgid);
    printf("Terminal control coordination completed.\n");

    kill(-pgid, SIGCONT);

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Process group completed successfully.\n");
        printf("Process group state validated.\n");
    }

    return 0;
}
|
root@HP:~/skill_week10# ./task4_process_groups
Process Group Demonstration
Parent PID: 659
Child PID: 660
Process Group ID: 660
Child process group created successfully.
Child process group ID retrieved: 660
Terminal control coordination completed.
Child process completed.
Process group completed successfully.
Process group state validated.
root@HP:~/skill_week10# |

```

Observation :- Process suspension, resumption, process groups, and signal coordination were successfully implemented and tested.